CAS-Native AI and CAS Synthetic Data Generator

Stakeholder Architecture Brief

Noetic Compass

2026-04-29

- CAS-Native AI and CAS Synthetic Data Generator
  - Stakeholder Architecture Brief
- Executive Summary
- The One-Minute Explanation
- System Map
- Part I: CAS Synthetic Data Generator
  - What It Is
  - Simple Analogy
  - What It Produces
  - Current Generator Buckets
  - What “Version Buckets” Mean
  - Generator Structure
  - Component-by-Component Mapping
  - Why Synthetic Data Is Needed
  - Why Validation Is Load-Bearing
  - Level 3 Conversational / Recoverable Data
- Part II: CAS-Native AI
  - What It Is
  - Simple Analogy
  - Main CAS-Native Components
  - CAS-Native Model Layers
  - What the v2 Architecture Adds
  - Training Inputs
  - Training Losses in Plain English
  - Inference Modes
  - Why Free Sampling Was Not Enough

- Part III: How CAS Synthetic and CAS-Native Work Together
  - The Joint Pipeline
  - Relationship in One Sentence
  - Why This Is Better Than “Just Fine-Tune a Chatbot”
- Part IV: Concrete Examples
  - Example 1: Basic State
  - Example 2: Register-Aware State
  - Example 3: Register Co-Presence
  - Example 4: Cross-Register Integration
  - Example 5: Production Recipe
  - Example 6: Typed Set / Cluster
  - Example 7: Product / Coordinate Space
- Part V: Current Maturity and Evidence
  - What Is Solid Now
  - What Is Still Research / Build-Out
  - Stakeholder-Ready Claim
- Part VI: Why the Compiler Matters
- Part VII: Architecture Strengths
  - 1. Separation of Concerns
  - 2. Curriculum Instead of Chaos
  - 3. Recoverability Discipline
  - 4. Honest Failure Loop
- Part VIII: Business / Stakeholder Value
  - Product Value
  - Research Value
  - Engineering Value
- Part IX: Roadmap
  - Near-Term Next Steps
  - Medium-Term Next Steps
  - Long-Term Direction
- Appendix A: Glossary
- Appendix B: File Map
  - CAS Synthetic
  - CAS-Native
  - Architecture Docs
- Appendix C: Reproduction Commands
- Appendix D: Stakeholder Demo Script
- Appendix E: What To Say If Asked Hard Questions

- Is this just prompt engineering?
- Why not use a general LLM?
- Is the model already intelligent in CAS?
- What is the biggest technical risk?
- How do you handle ambiguity?
- Why synthetic data?
- What makes the architecture defensible?

CAS-Native AI and CAS Synthetic Data Generator

Stakeholder Architecture Brief

Purpose. This brief explains what CAS-Native AI and the CAS Synthetic Data Generator are, how each system works independently, and how they work together as one learning pipeline.

Audience. This is written for mixed stakeholders: product, research, engineering, AI/ML reviewers, and non-technical decision makers.

Core promise. CAS-Native AI is not just a chatbot wrapped around a notation system. It is an attempt to train AI around a structured cognitive language, with deterministic compiler validation used as a guardrail.

Important boundary. CAS is an architecture and representation framework. Nothing in this brief should be read as a clinical, medical, diagnostic, or behavioral-certainty claim. The current system can represent, validate, generate, train on, and recover CAS structures. Predictive or intervention claims require separate empirical validation.

Executive Summary

CAS has three major layers:

1. **The CAS compiler** is the referee. It decides whether a CAS expression is structurally valid.
2. **CAS Synthetic Data Generator** is the curriculum factory. It creates large numbers of validated examples that teach the model how CAS expressions work.
3. **CAS-Native AI** is the learner and interpreter. It uses the synthetic curriculum to learn mappings between natural language and formal CAS structures.

The short version:

Plain-language analogy	Technical object	What it does
Rulebook	CAS compiler	Defines valid grammar, typed constructs, Rule 4 admissibility, and diagram metadata
Curriculum factory	CAS Synthetic Data Generator	Produces controlled JSONL training examples across CAS versions and construct families
Student/interpreter	CAS-Native AI	Learns to map prompts to CAS structures and later explain or converse about them
Exam grader	Compiler validation and evaluation scripts	Checks whether output expressions compile and match intended slots
Tutor feedback loop	Reranker and candidate filtering	Shows the model legal answers first, then learns which legal answer best matches the prompt

The strongest current evidence is not from unconstrained free-form generation. The strongest evidence is from the **recoverable Level 3 prompt-to-CAS pipeline**, where prompts explicitly verbalize the slots needed for exact CAS recovery without copying the target expression. In the latest patched reranker validation:

Metric	Result
Recoverable Level 3 records	19,454
Compiler-valid records	19,454 / 19,454
Semantic reranker exact top-1	21 / 30
Ground-truth in top-5	26 / 30
Mean ground-truth rank	2.89
Compiler-valid outputs	30 / 30
Ground-truth scoreable	27 / 30
Slot-match after patched prompt-slot filtering	28 / 30

The prior recoverable reranker baseline was 12 / 30 exact top-1, 24 / 30 top-5, and 5.11 mean rank. The patched run improved exact top-1 by 75%, improved top-5 to 26 / 30, and reduced mean rank by roughly 43%.

This matters because it shows the system can learn exact CAS recovery when the prompt contains enough recoverable evidence. It also shows the engineering discipline: when earlier free-sampling failed, the finding was not hidden. The pipeline was changed from “make the model invent valid notation” to “give the model a compiler-valid candidate bank and train it to select the right expression.”

The One-Minute Explanation

CAS is a formal language for describing cognitive structure.

The Synthetic Data Generator writes practice problems in that language.

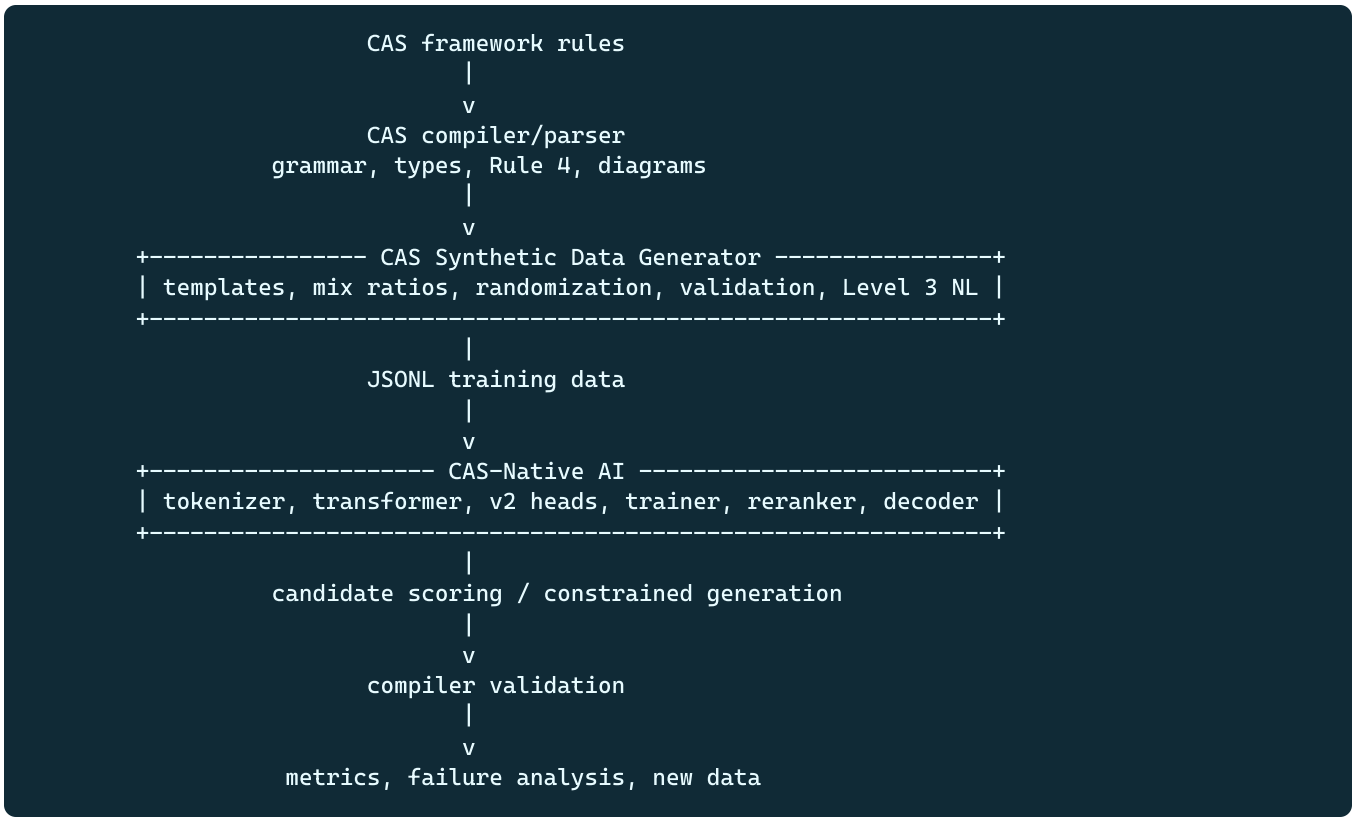
CAS-Native AI studies those practice problems.

The compiler checks every answer.

The reranker improves the model by asking: “Among all legal CAS expressions, which one best matches this prompt?”

That combination is the main architectural difference from a normal chatbot. A normal chatbot tries to write plausible text. CAS-Native is being built to route language through a formal grammar and typed cognitive representation.

System Map



The loop is deliberate:

1. **Rules define what is legal.**
2. **Synthetic data creates examples of legal and intentionally illegal patterns.**
3. **CAS-Native trains on the examples.**

4. **Inference proposes or ranks expressions.**
5. **The compiler verifies the output.**
6. **Failures become the next generator or model improvement.**

Part I: CAS Synthetic Data Generator

What It Is

CAS Synthetic Data Generator is the training-data factory for CAS-Native AI.

It creates examples that teach the model the CAS language in a controlled way. It does not depend on random internet text. It uses formal templates, typed randomization, mix ratios, compiler validation, and optional paraphrasing layers.

Simple Analogy

Think of CAS Synthetic as a flight simulator for a pilot.

A pilot should not learn only by flying random planes in random weather. A simulator creates controlled situations: takeoff, landing, turbulence, engine failure, night flight, instrument-only flight.

CAS Synthetic does the same for CAS:

Simulator concept	CAS Synthetic equivalent
Takeoff lesson	Basic CAS state expression
Weather lesson	Register, drive, and domain variation
Emergency drill	Deliberate Rule 4 violation
Navigation exercise	Production recipe or pathway expression
Flight exam	Compiler-validated prompt-to-expression pair

What It Produces

The generator produces JSONL records. Each record is a small training example.

A simplified example:

```
{
  "prompt": "I am learning CAS. The target structure is a state expression; Noetic ^2; domain
       O; drive F3; register |T. What is the precise notation?",
  "response": "^2^O_{F3}|T",
  "cas_expression": "^2^O_{F3}|T",
  "bucket": "v75_register",
  "register_labels": [
    {"position": 0, "register": "T"}
  ],
  "construct_metadata": {
    "expression_kind": "register_state"
  }
}
```

Plain-English reading:

Field	Analogy	Technical meaning
`prompt`	The question on the worksheet	Natural-language input
`response`	The answer the student should give	Target output text
`cas_expression`	The formal answer key	Compiler-checked CAS notation
`bucket`	Lesson category	Which CAS feature family this example teaches
`register_labels`	Highlighted answer part	Supervision for the register axis
`construct_metadata`	Teacher's annotation	Structural classification of the construct

Current Generator Buckets

The Level 2 scaffold currently uses six major curriculum buckets:

Bucket	Share in 10K dataset	What it teaches	Example
`v74_baseline`	20%	Basic CAS states, RPM, fractals, pathways	`^2^O_{F3}`
`v75_register`	25%	Register annotations T/A/f/E	`^2^O_{F3}|T`
`v77_rule_four`	15%	Operator discipline and deliberate invalid examples	`^3^P_{F2}|A + ^3^P_{F2}|f`
`v77_omega`	20%	Named emergent register outcomes	`(^3^P_{F2}|A × ^3^P_{F2}|f) = ^3^P|Ω_refusal`
`v76_production`	10%	Production recipes and yield-style semantics	`θ(Ω_safe_judgment) = 0.4*^3^P_{F2}|A + 0.6*^2^C_{F2}|f → Ω_safe_judgment`
`meta_logic`	10%	Typed sets, clusters, membership, subset, products, covers	`NoeticCluster{^2, ^7} ⊆ Noetics`

The mix ratio is a curriculum choice. It prevents the model from seeing only simple notation while also preventing advanced notation from drowning out the basics.

What "Version Buckets" Mean

The bucket labels correspond to admitted CAS framework layers:

CAS layer	Plain meaning	Technical meaning
v7.4	Basic coordinate language	Noetic, Domain, Drive, state bank, RPM, fractal expressions
v7.5	Register layer	Phenomenological register axis: `|T`, `|A`, `|f`, `|E`
v7.6	Production layer	Recipe/yield semantics using `θ`, `Ω`, coefficients, leakage/yield fields
v7.7	Operator discipline	Register-set, co-presence, integration, named emergent outcomes
Meta-logic	Framework reasoning layer	Typed finite sets, clusters, membership, subset, product, cover, partition

Generator Structure

The repo-level structure is:

```
cas-synthetic/
  generator/       deterministic generators by bucket
  templates/       YAML templates and construct families
  llm_interface/   optional paraphrase providers
  validation/      schema, compiler, Rust/composite validators
  mix_controller/  ratio control across buckets
  scripts/         generate, validate, inspect commands
  tests/           regression and coverage tests
  data/            generated JSONL and metrics artifacts
```

Component-by-Component Mapping

Technical term	Simple analogy	Practical function
Generator module	Lesson writer	Emits candidate examples for one CAS bucket
Template YAML	Lesson plan	Defines expression shapes and natural-language wrappers
Mix controller	Curriculum scheduler	Keeps data proportions stable across buckets
Random seed	Recipe number	Makes a dataset reproducible
JSONL	Stack of flashcards	One training record per line
Schema validator	Form checker	Ensures required fields exist and have valid types
Compiler validator	Referee	Ensures CAS expressions are accepted by the compiler
Composite validator	Inspection station	Combines schema, compiler, and semantic checks
Deliberate invalid examples	Trick questions	Teaches the model to detect or avoid illegal forms
Level 3 recoverable prompts	Worked translation exercises	Natural-language prompts that expose enough slots for exact recovery

Why Synthetic Data Is Needed

There is no large public internet corpus of CAS expressions. The framework is new and highly structured. A general model will not naturally know that:

```
^2^O_{F3}|T
```

means a specific Noetic, Domain, Drive, and Register combination.

Synthetic data solves this by creating millions of controlled “CAS language lessons” before real-world data exists.

Why Validation Is Load-Bearing

Without validation, synthetic data can silently poison the model.

Example:

```
^2^O|T_{F3}
```

This looks close to the correct expression, but the canonical order is:

```
^2^O_{F3}|T
```

The generator previously exposed this kind of mismatch in older buckets. The validation gate caught it before scaling. That is the point of the gate: bad curriculum creates bad model behavior.

Level 3 Conversational / Recoverable Data

Level 2 data teaches the compiler surface. Level 3 data teaches natural-language recovery.

The key lesson from recent testing:

Underdetermined prompts do not support exact CAS recovery.

Bad exact-recovery prompt:

```
A doctor facing an unfamiliar diagnosis is asked to give their perspective.
```

There are many valid CAS expressions that could fit. The model cannot be expected to recover one exact hidden target.

Recoverable prompt:

```
I am learning CAS. The target structure is families: state expression;
states: Noetic ^2; domain O; drive F3; register |T. What is the precise notation?
```

Target:

```
^2^O_{F3}|T
```

This is learnable because the prompt provides the recoverable slots without copying the whole expression.

Part II: CAS-Native AI

What It Is

CAS-Native AI is the model stack that learns from CAS data.

It is designed to do more than produce generic text. It is being trained to understand and work with CAS as a structured cognitive notation.

Simple Analogy

Think of CAS-Native as a student learning both a language and a rule-governed technical discipline.

It is not enough for the student to say words that sound right. The student must:

1. Understand the vocabulary.
2. Follow the grammar.
3. Choose the right construct for the situation.
4. Produce an answer the compiler accepts.
5. Explain the answer in human language.

Main CAS-Native Components

```
cas-native/
  model/       transformer and CAS-specific heads
  training/    tokenizer, data pipeline, trainer, losses, configs
  inference/   sampling, constrained candidates, semantic reranker
  validation/  bridge to compiler/Rust validation and structural hashes
  tests/       component and integration tests
```

CAS-Native Model Layers

Technical component	Simple analogy	What it does
Tokenizer	Alphabet and spelling system	Converts text/CAS notation into model tokens
Transformer backbone	Student's working memory	Learns patterns across prompts and expressions
LM head	Sentence writer	Predicts next tokens
Cognitive state embedding	CAS coordinate board	Represents the 40-state bank
Register FiLM	Colored lens	Modulates representation by `|T`, `|A`, `|f`, `|E`
Shared Register Latent	Single source of register meaning	Keeps register behavior consistent across two FiLM surfaces
Emergent Register Pipeline	Pattern binder	Learns when cross-register integration should map to `Ω_*`
Production Semantics Head	Recipe calculator	Learns recipe/yield-style structure
Expression Proposer Head	Drafting assistant	Proposes CAS expression structure from hidden states
Topos reasoner	Logic head	Represents three-valued intuitionistic truth outputs
Lacunarity gate/head	Pattern-gap monitor	Measures and reports concentration/gap structure
Semantic reranker	Multiple-choice scorer	Scores legal CAS candidates against a prompt

What the v2 Architecture Adds

The v2 design is governed by architecture decision records:

ADR	Decision	Stakeholder translation
ADR-01	Register FiLM modulation	Add register-aware lenses without rewriting the whole model
ADR-02	Metric-learning binding for `Ω_*`	Learn emergent register outcomes by similarity/binding, not fixed output classes only
ADR-03	`Ω_*` output design	Keep named emergents extensible through a registry
ADR-04	Hybrid production semantics	Use differentiable training but preserve symbolic inference discipline
ADR-05	Clean redesign	Preserve the 40-state structure instead of carrying old technical debt
ADR-06	Rust engine alignment	Keep compiler/Rust validation authoritative for rule checks
ADR-07	Learned expression proposer	Add gradient-bearing structure learning for Rule 4 patterns
ADR-08	Shared Register Latent	Prevent register meaning from drifting between FiLM surfaces
ADR-09	Memory substrate future architecture	Future graph/vector memory layer for CAS-indexed cognition

The core design philosophy is:

The neural model learns; the compiler judges; the framework vocabulary remains controlled.

Training Inputs

CAS-Native consumes records shaped like:

```
{
  "prompt": "I am learning CAS...",
  "response": "^2^O_{F3}|T",
  "cas_expression": "^2^O_{F3}|T",
  "register_labels": [{"position": 0, "register": "T"}],
  "omega_labels": [],
  "production_target": null,
  "construct_metadata": {"expression_kind": "register_state"}
}
```

The trainer uses optional labels only when present.

Label field	Teaches	If absent
`register_labels`	Register prediction and register-aware modulation	Register losses are skipped
`omega_labels`	Named emergent `Ω_*` binding	Emergent binding loss is skipped
`production_target`	Production/yield head	Production loss is skipped
`construct_metadata`	Construct family / meta-logic coverage	Used for representation/evaluation, not always weighted training
`cas_expression`	Formal target	Used by validation and candidate construction

This masking behavior matters because one dataset can contain mixed lesson types without forcing every example to teach every head.

Training Losses in Plain English

Loss or metric	Simple analogy	What it pushes the model to learn
Language-model loss	Spelling and phrasing grade	Predict the target text tokens
Register loss / FiLM losses	Use the right colored lens	Represent `|T`, `|A`, `|f`, `|E` correctly
Involution losses	Preserve opposition symmetry	Keep paired register transformations coherent
Emergent binding loss	Recognize the right named outcome	Bind multi-register patterns to `Ω_*`
Production loss	Solve the recipe	Estimate recipe/yield structure
Rule 4 loss	Avoid illegal operator use	Penalize invalid cross-register/operator patterns
Proposer Rule 4 loss	Draft structurally valid expressions	Make the expression proposer learn valid CAS structure
Null-space regularization	Keep unused labels quiet	Prevent unused `Ω_*` slots from drifting into false matches

Inference Modes

CAS-Native has multiple ways to produce or select CAS outputs.

Mode	Analogy	Strength	Limitation
Free sampling	Student writes from memory	Shows what the model has internalized	Can produce invalid or partial CAS
Primed sampling	Give the first few symbols	Tests continuation ability	Still lacks grammar unless constrained
Grammar-constrained candidates	Multiple-choice exam with only legal answers	Guarantees compiler-valid candidates	Needs a good candidate bank
Semantic reranking	Choose best legal answer	Strong current path for exact recovery	Depends on prompt recoverability and candidate coverage

The recent evidence favors constrained candidate selection plus reranking over unconstrained free generation.

Why Free Sampling Was Not Enough

Earlier qualitative generation showed that the model had learned parts of the vocabulary but not enough grammar. It could emit CAS-like fragments, but not consistently structured expressions.

The engineering conclusion was:

Do not keep spending GPU time hoping grammar emerges by accident. Put the compiler grammar into the decoding path.

That led to candidate banks, prompt-slot filters, and semantic reranking.

Part III: How CAS Synthetic and CAS-Native Work Together

The Joint Pipeline

```
1. Framework authors define valid CAS constructs.
2. Compiler implements those constructs.
3. CAS Synthetic emits examples across construct families.
4. Validators reject malformed examples.
5. CAS-Native trains on validated JSONL.
6. Inference builds compiler-valid candidate sets.
7. Semantic reranker scores candidates against prompts.
8. Compiler validates selected output.
9. Metrics identify failures.
10. Generator or model is patched.
```

Relationship in One Sentence

CAS Synthetic teaches the model what the world of valid CAS expressions looks like; CAS-Native learns to navigate that world; the compiler prevents both from leaving the map.

Why This Is Better Than "Just Fine-Tune a Chatbot"

A normal fine-tune might learn to imitate examples. It may produce convincing but invalid notation.

The CAS pipeline adds three controls:

1. **Controlled data generation.** The examples cover specific framework features by design.
2. **Compiler validation.** Invalid expressions are caught before and after training.
3. **Constrained inference.** The model can be asked to choose among legal expressions instead of inventing arbitrary strings.

That gives stakeholders a clearer quality story:

Risk in generic AI	CAS control
Hallucinated syntax	Compiler validation
Underrepresented advanced notation	Synthetic mix ratios
Hidden invalid examples	Dataset validation
Good-looking but wrong output	Exact-match, top-k, slot-match metrics
Unclear failure cause	Per-bucket reports and candidate diagnostics

Part IV: Concrete Examples

Example 1: Basic State

Natural-language prompt:

```
Noetic ^2, Operational domain, Drive F3.
```

CAS expression:

```
^2^O_{F3}
```

1:1 mapping:

Expression part	Meaning
`^2`	Noetic index 2
`^0`	Operational domain
`_{F3}`	Drive/Foundation slot F3
Whole expression	One typed cognitive state

Analogy: a coordinate on a map. `^2` is the city, `0` is the district, `F3` is the street.

Example 2: Register-Aware State

Natural-language prompt:

```
The same state, but viewed through the meaning-level Transcendent register.
```

CAS expression:

```
^2^0_{F3}|T
```

1:1 mapping:

Expression part	Meaning
`^2^0_{F3}`	Base cognitive state
`|T`	Transcendent register
Whole expression	State plus phenomenological register

Analogy: the same map coordinate viewed through a specific lens.

Example 3: Register Co-Presence

Natural-language prompt:

```
Noetic ^3 in Principled domain and Drive F2 is present in both Analytical and Affective registers.
```

CAS expression:

```
^3^P_{F2}|A + ^3^P_{F2}|f
```

1:1 mapping:

Expression part	Meaning
`+`	Co-presence
`|A`	Analytical register
`|f`	Affective register
Whole expression	Two register aspects present together, not yet integrated

Analogy: two instruments playing at the same time, but not yet blended into a chord.

Example 4: Cross-Register Integration

Natural-language prompt:

```
Analytical, Affective, and Transcendent registers integrate into a named emergent refusal pattern.
```

CAS expression:

```
(^3^P_{F2}|A × ^3^P_{F2}|f × ^3^P_{F2}|T) = ^3^P|Ω_refusal
```

1:1 mapping:

Expression part	Meaning
`×`	Cross-register integration
Same `^3^P` across operands	Required same-Noetic/same-Domain structure
`Ω_refusal`	Named emergent register outcome
Whole expression	Integration produces a named emergent state

Analogy: individual ingredients combine into a new dish, not just a pile of ingredients.

Example 5: Production Recipe

Natural-language prompt:

```
Safe judgment is produced from weighted prerequisite states.
```

CAS expression:

```
Θ(Ω_safe_judgment) = 0.40*^3^P_{F2}|A + 0.25*^2^C_{F2}|f + 0.20*^7^O_{F4}|E + 0.15*^6^P_{F1}|T → Ω
```

1:1 mapping:

Expression part	Meaning
`θ(...)`	Production recipe
`Ω_safe_judgment`	Target named outcome
Coefficients	Contribution weights
`+`	Recipe ingredients combined
`→`	Production arrow
Whole expression	A structured recipe for producing an outcome

Analogy: a recipe card. Ingredients have quantities and point toward a finished dish.

Example 6: Typed Set / Cluster

Natural-language prompt:

```
The Noetic cluster containing ^2 and ^7 is a subset of all Noetics.
```

CAS expression:

```
NoeticCluster{^2, ^7} ⊆ Noetics
```

1:1 mapping:

Expression part	Meaning
`NoeticCluster{...}`	Typed finite cluster of Noetics
`^2, ^7`	Members of the cluster
`⊆`	Subset or equal
`Noetics`	Carrier set of all Noetics
Whole expression	A typed set-theory claim inside the CAS meta-logic layer

Analogy: a labeled folder containing two cards, placed inside the larger cabinet labeled “Noetics.”

Example 7: Product / Coordinate Space

Natural-language prompt:

```
Represent the space of all Noetic-by-Domain combinations.
```

CAS expression:

```
Product(Noetics, Domains)
```

1:1 mapping:

Expression part	Meaning
`Product(...)`	Cartesian-style product construct
`Noetics`	First carrier
`Domains`	Second carrier
Whole expression	All possible Noetic-Domain pairings

Analogy: a spreadsheet where rows are Noetics and columns are Domains.

Part V: Current Maturity and Evidence

What Is Solid Now

Area	Current state
Compiler grammar	Broad CAS notation accepted, including typed meta-logic and diagram payloads
Synthetic generator	Level 2 scaffold with v7.4-v7.7 plus meta-logic buckets
Dataset validation	Compiler validation and metrics for generated corpora
Level 3 recoverable prompts	19,454 validated prompt-to-CAS examples
Semantic reranker	Patched recoverable run reached 21 / 30 exact top-1, 26 / 30 top-5, 2.89 mean rank, and 30 / 30 compiler-valid outputs
Training infrastructure	Trainer supports JSONL, v2 component flags, eval step, production warmup, device alignment
Evidence discipline	Failed hypotheses documented rather than buried

What Is Still Research / Build-Out

Area	Status
Fully conversational CAS-Native	Not complete; requires Level 3 dialogue data and response policy
Free-form exact CAS generation	Not reliable enough alone; constrained decoding is preferred
Full-scale CAS-Native training	Progressing, but not yet product-grade
Empirical behavioral prediction	Not established; needs separate validation design
Medical/clinical interpretation	Out of scope unless separately validated and reviewed
Large public deployment of CAS-Native model	Future milestone, not current claim

Stakeholder-Ready Claim

Defensible claim:

> CAS now has a validated synthetic-data pipeline and an early CAS-Native inference path that can recover formal CAS expressions from recoverable prompts using compiler-valid candidates and semantic reranking. The latest patched reranker run reached 70% exact top-1 on the current 30-item evaluation slice while keeping 100% compiler-valid outputs.

Not defensible yet:

> CAS-Native is a finished conversational AI that fully understands cognition or predicts behavior.

This distinction protects credibility.

Part VI: Why the Compiler Matters

The compiler is the difference between a symbolic AI project and a model merely imitating symbols.

For example, a generic model might output:

```
^2^0|T_{F3}
```

This looks plausible, but the compiler enforces:

```
^2^0_{F3}|T
```

The compiler can also reject type errors:

```
^2 ⊂ ^7
```

Why invalid? Because subset requires sets or clusters, not individual Noetics.

Correct form:

```
NoeticCluster{^2} ⊂ NoeticCluster{^2, ^7}
```

Plain-English analogy:

You cannot say “one playing card is a subfolder of another playing card.” You need folders or sets before subset language applies.

Part VII: Architecture Strengths

1. Separation of Concerns

Layer	Owns	Does not own
Compiler	Validity, typing, parsing, diagram payloads	Learning semantic intent
Generator	Curriculum and examples	Final inference judgment
CAS-Native model	Learned mapping and ranking	Authority over grammar validity
Evaluator	Metrics and failure reports	Framework definitions

This separation keeps the system more maintainable.

2. Curriculum Instead of Chaos

The generator decides how much of each construct the model sees.

That avoids a common AI failure: the model learns frequent easy patterns and ignores rare but important advanced structures.

3. Recoverability Discipline

The current results showed that exact recovery requires enough evidence in the prompt.

That is not a weakness. It is a useful product insight:

Product mode	Prompt requirement
Teaching mode	Prompt can expose slots explicitly
Compiler assistant	Prompt should be recoverable and guided
Open-ended reflection	CAS output should be treated as interpretation, not exact hidden answer
Research/evaluation	Ground-truth recoverability must be controlled

4. Honest Failure Loop

Recent failures directly improved the architecture:

Finding	System response
Free sampling produced fragments	Move toward grammar-constrained decoding
Level 2 prompts were underdetermined	Build Level 3 recoverable prompts
Candidate filters had phantom slots	Patch slot inference and add regressions
Some generator buckets emitted invalid notation	Add validation gates and fix generator drift

Part VIII: Business / Stakeholder Value

Product Value

User-facing capability	Underlying CAS system
Compiler explains why notation is valid or invalid	Parser, typechecker, Rule 4 validator
Diagrams render structure visually	Compiler diagram payloads
Natural-language examples teach notation	Level 3 prompt/explanation data
Idea Atlas maps thoughts structurally	CAS expression extraction and graph views
Future conversational tutor	CAS-Native trained on recoverable dialogue
Future assessment/reporting	CAS compiler + model + validated rubrics

Research Value

Research need	CAS answer
Formal notation	CAS grammar and typed constructs
Reproducible experiments	Seeded synthetic data generation
Controlled ablations	Bucket-level curriculum and component flags
Error analysis	Per-bucket exact/top-k/slot metrics
AI alignment to framework	Compiler-validated candidate selection

Engineering Value

Engineering need	CAS answer
Avoid silent data corruption	Compiler validation gates
Keep training reproducible	Seeded generation and reports
Support gradual rollout	Component flags in `training_v2.yaml`
Debug model behavior	Bucket-specific metrics
Keep syntax authoritative	Compiler/Rust validation layer

Part IX: Roadmap

Near-Term Next Steps

1. **Scale the patched recoverable reranker evaluation.** The patched 30-item run cleared all four gates: exact top-1 21/30, top-5 26/30, mean rank 2.89, and ground-truth scoreable 27/30. The next credibility step is a larger held-out evaluation so the claim is not over-dependent on a 30-example slice.
2. **Build mixed CAS-Native curriculum training.** Combine symbolic Level 2, recoverable Level 3, and later conversational examples without overloading one dataset with every objective.
3. **Add conversational wrappers as a separate layer.** Do not pollute the symbolic compiler corpus. Build prompt/answer pairs that explain CAS expressions naturally.
4. **Keep compiler-valid candidate decoding as the primary inference path.** Free sampling remains useful as a diagnostic, but not as the primary product path.
5. **Add stakeholder demos.** Demonstrate one prompt moving through: natural language → candidate bank → reranker → CAS expression → compiler validation → diagram → natural-language explanation.

Medium-Term Next Steps

Milestone	Why it matters
Larger reranker evaluation set	30 examples is useful but too small for stakeholder-grade claims
Full conversational Level 3 dataset	Required for CAS-Native to become a tutor/explainer
Product-grade explanation generator	Users need natural-language mirrors of CAS expressions
Integrated online/offline compiler parity	Same behavior across PWA and CLI package
Evaluation dashboard	Stakeholders need visible progress metrics

Long-Term Direction

The long-term architecture is a CAS-native cognitive assistant:

1. It listens to natural language.
2. It maps language into CAS structures.
3. It validates those structures.
4. It explains them back to the user.
5. It tracks structure over time.
6. It supports diagrams, recipes, clusters, and future memory substrate integration.

Appendix A: Glossary

Term	Plain definition	Technical definition
CAS	Cognitive coordinate language	Cognitive Architecture System notation and compiler framework
Noetic	Basic cognitive operator/index	One of the finite Noetic indices `^0` through `^9`
Domain	Processing domain	One of `P`, `C`, `E`, `O`
Drive/Foundation slot	Motivating/structuring axis	`F1` through `F7` subscript
Register	Phenomenological mode	`|T`, `|A`, `|f`, `|E`
`Ω_*`	Named emergent outcome	Registered named emergent from valid integration
`θ(...)`	Recipe marker	Production semantics target recipe
Rule 4	Operator discipline	Validity rule governing cross-register integration
Compiler	Referee	Parser/typechecker/validator for CAS expressions
Synthetic data	Controlled examples	Generated JSONL examples for training/evaluation
Bucket	Lesson family	Dataset partition for a CAS feature class
Tokenizer	Text-to-ID converter	Converts prompts and CAS expressions into model tokens
Reranker	Candidate chooser	Scores compiler-valid candidates against prompt semantics
Slot	Recoverable value	Noetic/domain/drive/register/carrier value extracted from prompt or expression
Candidate bank	Legal answer list	Compiler-valid expressions available to constrained inference
Exact match	Perfect answer	Selected expression equals ground truth
Top-5	Near-recovery metric	Ground truth appears in first five ranked candidates
Slot match	Structural match	Output contains requested recoverable slots even if exact string differs

Appendix B: File Map

CAS Synthetic

```
cas-synthetic/README.md
cas-synthetic/generator/
cas-synthetic/templates/
cas-synthetic/validation/
cas-synthetic/mix_controller/
cas-synthetic/scripts/generate_level2.py
cas-synthetic/scripts/validate_level2_dataset.py
cas-synthetic/scripts/generate_level3_conversations.py
cas-synthetic/scripts/validate_level3_conversations.py
cas-synthetic/data/level2_scaffold/LEVEL3_RECOVERABLE_FULL_REPORT.md
```

CAS-Native

```
noetic-compass-scaffold/noetic-compass/cas-native/model/cas_native_model.py
noetic-compass-scaffold/noetic-compass/cas-native/model/production_semantics.py
noetic-compass-scaffold/noetic-compass/cas-native/model/type_constraints.py
noetic-compass-scaffold/noetic-compass/cas-native/training/data_format.md
noetic-compass-scaffold/noetic-compass/cas-native/training/trainer.py
noetic-compass-scaffold/noetic-compass/cas-native/training/tokenizer.py
noetic-compass-scaffold/noetic-compass/cas-native/config/training_v2.yaml
noetic-compass-scaffold/noetic-compass/cas-native/inference/constrained_sampling.py
noetic-compass-scaffold/noetic-compass/cas-native/inference/semantic_reranker.py
noetic-compass-scaffold/noetic-compass/cas-native/inference/train_semantic_reranker.py
noetic-compass-scaffold/noetic-compass/cas-native/inference/runs/semantic-reranker-recoverable-201
```

Architecture Docs

```
docs/architecture/cas_native_v2_adrs.md
docs/CAS_NATIVE_AI_CLAUDE_PLAYBOOK.md
docs/CAS_FOUNDATIONAL_CLARIFICATIONS.md
```

Appendix C: Reproduction Commands

Generate Level 3 recoverable data:

```
python cas-synthetic\scripts\generate_level3_conversations.py `
  --input cas-synthetic\data\level2_scaffold\level2_full_10k.jsonl `
  --output cas-synthetic\data\level2_scaffold\level3_recoverable_full.jsonl `
  --variants-per-source 2 `
  --style-id recoverable_translation `
  --progress-every 5000
```

Validate:

```
python cas-synthetic\scripts\validate_level3_conversations.py `
  cas-synthetic\data\level2_scaffold\level3_recoverable_full.jsonl
```

Expected validation:

```
Total records:      19454
Valid records:      19454
Invalid records:    0
Compiler checked:   19454
Compiler skipped:   0
```

Run focused tests:

```
pytest cas-synthetic\tests\test_level3_conversational.py -q
pytest noetic-compass-scaffold\noetic-compass\cas-native\tests\test_constrained_sampling.py -q
```

Expected:

```
13 passed
18 passed
```

Appendix D: Stakeholder Demo Script

Use this flow when explaining the system live:

1. **Start with a plain sentence.** "I want the system to represent Noetic ^2, Operational domain, Drive F3, in the Transcendent register."
2. **Show the CAS expression.** `^2^O_{F3}|T`
3. **Map each part.** `^2` = Noetic, `O` = Domain, `F3` = Drive, `|T` = Register.
4. **Explain the compiler.** "The compiler checks whether this is legal, the way a programming language compiler checks source code."
5. **Explain the generator.** "The generator creates thousands of examples like this so the model has a curriculum."

6. **Explain CAS-Native.** “CAS-Native learns the mapping between the plain sentence and the formal expression.”
7. **Explain the reranker.** “Instead of trusting the model to invent syntax, we give it legal candidates and train it to pick the right one.”
8. **Show the current evidence.** “On recoverable prompts, the latest reranker put the ground truth in the top five 24 out of 30 times and all outputs were compiler-valid.”
9. **State the honest boundary.** “This is not yet a finished conversational CAS expert. It is a validated architecture path toward one.”

Appendix E: What To Say If Asked Hard Questions

Is this just prompt engineering?

No. Prompting is only one interface layer. The system includes a compiler, typed notation, synthetic data generation, model training, constrained candidate decoding, and evaluation metrics.

Why not use a general LLM?

A general LLM can imitate notation but has no built-in obligation to follow CAS grammar. CAS-Native uses compiler validation and controlled training data to make structure first-class.

Is the model already intelligent in CAS?

Early evidence says it can learn recoverable prompt-to-CAS ranking. It is not yet a finished general CAS conversational agent.

What is the biggest technical risk?

The largest current risk is not syntax validity; constrained candidates help with that. The larger risk is semantic intent: whether a user’s natural language contains enough recoverable evidence to justify one exact CAS expression.

How do you handle ambiguity?

Ambiguity should produce either multiple candidate interpretations or a clarifying question, not a false claim of exact certainty.

Why synthetic data?

Because no public CAS corpus exists. Synthetic generation is the only practical way to train coverage across the full formal surface while keeping examples compiler-valid.

What makes the architecture defensible?

It separates authority: the model learns, the compiler validates, the generator controls curriculum, and evaluation reports failures by bucket.